

Optymalizacja Kodu Na Różne Architektury

Ćwiczenie 3

1 Wprowadzenie

Rozważmy następującą procedurę uruchamiającą eliminację Gaussa bez pivotingu

Listing 1: Bazowa wersja kodu

```
//requires additional changes to the code to make it work

#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <time.h>

static double gtod_ref_time_sec = 0.0;

double ge_flop(int SIZE)
{
    double n = (double) SIZE;
    return (n - 1.0) * n * (2.0 * n - 1.0) / 2.0;
}

/* Adapted from the bl2_clock() routine in the BLIS library */

double dclock()
{
    double the_time, norm_sec;
    struct timeval tv;
    gettimeofday( &tv, NULL );
    if ( gtod_ref_time_sec == 0.0 )
```

```

    gtod_ref_time_sec = ( double ) tv.tv_sec;
    norm_sec = ( double ) tv.tv_sec - gtod_ref_time_sec;
    the_time = norm_sec + tv.tv_usec * 1.0e-6;
    return the_time;
}

```

```

int ge(double ** A, int SIZE)
{
    int i,j,k;
    for (k = 0; k < SIZE; k++) {
        for (i = k+1; i < SIZE; i++) {
            for (j = k+1; j < SIZE; j++) {
                A[i][j] = A[i][j]-A[k][j]*(A[i][k]/A[k][k]);
            }
        }
    }
    return 0;
}

```

```

int main( int argc, const char* argv[] )
{
    int i,j,k,iret;
    double dtime;
    int SIZE = atoi(argv[1]);
    double matrix[SIZE][SIZE]; // TODO - make near optimal dynamic allocation
    srand(1);
    for (i = 0; i < SIZE; i++) {
        for (j = 0; j < SIZE; j++) {
            matrix[i][j] = rand();
        }
    }
    printf("call GE");
    dtime = dclock();
    iret = ge(matrix,SIZE);
    dtime = dclock()-dtime;
    printf("Time: %le\n", dtime);
    printf("FLOP: %le\n", ge_flop(SIZE));
    printf("GFLOP/s: %le\n", ge_flop(SIZE) / dtime / 1.0e9);

    double check=0.0;
    for (i = 0; i < SIZE; i++) {
        for (j = 0; j < SIZE; j++) {
            check = check + matrix[i][j];
        }
    }
}

```

```

    }
}
printf( "Check: %le\n", check );
fflush( stdout );

return iret;
}

```

2 Ćwiczenia

Przy kolejnych krokach proszę wykonywać kopie aktualnych wersji stanu kodu. Będą potrzebne przy ostatnim punkcie oraz na kolejnych zajęciach. Proszę na bieżąco notować czasy wykonywania się obliczeń.

2.1

Proszę wprowadzić zaznaczone w kodzie modyfikacje a następnie skompilować procedurę bez optymalizacji. Dla zadanego rozmiaru macierzy w kolejnych uruchomieniach wartość zmiennej **check** nie powinna ulegać zmianie.

```

maciekw@Banach:~ lab3> gcc ge1.c
maciekw@Banach:~ lab3> ./a.out 1500
call GETime: 1.314771e+00
FLOP: 3.371626e+09
GFLOP/s: 2.564421e+00
Check: -4.605781e+16
maciekw@Banach:~/studenci/OORA/skrypty/lab3>

```

2.2

Proszę wykonać kopię procedury

```
maciekw@Banach:~ lab3> cp ge1.c ge2.c
```

Proszę umieścić w rejestrach liczniki pętli.

Proszę skompilować procedurę z optymalizacją.

Jaki jest czas działania?

Jakie GFLOP/s uzyskuje procesor?

2.3

Proszę wykonać kopię procedury

```
maciekw@Banach:~ lab3> cp ge2.c ge3.c
```

Proszę umieścić w rejestrze powtarzającą się w najbardziej zagnieżdżonym miejscu wartość.

```
multiplier = (A[i][k]/A[k][k]);
```

Proszę skompilować procedurę z optymalizacją.

Jaki jest czas działania?

Jakie GFLOP/s uzyskuje procesor?

2.4

Proszę wykonać kopię procedury

```
maciekw@Banach:~ lab3> cp ge3.c ge4.c
```

Proszę ręcznie rozwinąć najbardziej zagnieżdżoną pętlę do 8 iteracji.

```
for (j = k+1; j < SIZE; ) {  
    if (j < (max(SIZE - BLKSIZE, 0))) {  
        //PROSZE UZUPELNIC  
        j += BLKSIZE;  
    }  
    else {  
        A[i][j]=A[i][j]-A[k][j]*multiplier;  
        j++;  
    }  
}
```

Proszę skompilować procedurę z optymalizacją.

Jaki jest czas działania?

2.5

Proszę wykonać kopię procedury

```
maciekw@Banach:~ lab3> cp ge4.c ge5.c
```

Proszę zamienić deklarację macierzy na macierz jednowymiarową indeksowaną poprzez makro (patrz laboratorium 2).

Proszę skompilować procedurę z optymalizacją.

Jaki jest czas działania?

Jakie GFLOP/s uzyskuje procesor?

2.6

Proszę wykonać kopię procedury

```
maciekw@Banach:~lab3> cp ge5.c ge6.c
```

Proszę wprowadzić blokowanie pamięci podręcznej oraz pakowanie danych do małych buforów roboczych. Całość wykonujemy analogicznie do <https://github.com/flame/how-to-optimize-gemm>

1. blokowanie zakresu aktualizowanej macierzy,
2. pakowanie intensywnie używanych danych do buforów ciągłych,
3. aktualizacja bloku za pomocą małego kernela obliczeniowego.

Proszę dodać następujące definicje rozmiarów bloków:

```
#define MR 8
```

```
#define NR 8
```

```
#define MC 128
```

```
#define NC 256
```

Znaczenie parametrów jest następujące:

- **NC** – liczba kolumn w pakowanym fragmencie wiersza pivota,
- **MC** – liczba wierszy aktualizowanych w jednym bloku,
- **NR** – liczba kolumn przetwarzanych jednocześnie w ręcznie rozwiniętym kernelu.

Następnie proszę utworzyć funkcję pakującą fragment wiersza pivota:

```
static void PackPivotRow(int n, const double *src, double *dst)
{
    int j;

    for (j = 0; j < n; j++) {
        dst[j] = src[j];
    }
}
```

Funkcja ta kopiuje ciągły fragment wiersza $A[k][j]$ do lokalnego bufora roboczego. Spakowany fragment powinien być następnie użyty wiele razy dla różnych bloków wierszy.

Proszę utworzyć funkcję pakującą mnożniki eliminacji:

```

static void PackMultipliers(int m, const double *A, int SIZE,
                           int row0, int k, double pivot,
                           double *mult)
{
    int i;

    for (i = 0; i < m; i++) {
        mult[i] = A[IDX(row0 + i, k, SIZE)] / pivot;
    }
}

```

Funkcja ta oblicza wartości:

$A[i][k] / A[k][k]$

dla małego bloku wierszy i zapisuje je w lokalnym buforze.

Następnie proszę utworzyć kernel aktualizujący blok macierzy:

```

static void UpdateKernel(int m, int n, double *C,
                        int ldc, const double *pivot_row,
                        const double *mult)
{
    int i, j;

    for (j = 0; j + NR <= n; j += NR) {
        register double p0 = pivot_row[j];
        register double p1 = pivot_row[j + 1];
        register double p2 = pivot_row[j + 2];
        register double p3 = pivot_row[j + 3];
        register double p4 = pivot_row[j + 4];
        register double p5 = pivot_row[j + 5];
        register double p6 = pivot_row[j + 6];
        register double p7 = pivot_row[j + 7];

        for (i = 0; i < m; i++) {
            double *c = C + i * ldc + j;
            register double mi = mult[i];

            c[0] = c[0] - p0 * mi;
            c[1] = c[1] - p1 * mi;
            c[2] = c[2] - p2 * mi;
            c[3] = c[3] - p3 * mi;
            c[4] = c[4] - p4 * mi;
            c[5] = c[5] - p5 * mi;
            c[6] = c[6] - p6 * mi;
            c[7] = c[7] - p7 * mi;
        }
    }
}

```

```

    }
}

for (; j < n; j++) {
    register double pj = pivot_row[j];

    for (i = 0; i < m; i++) {
        C[i * ldc + j] = C[i * ldc + j] - pj * mult[i];
    }
}
}

```

Po dodaniu funkcji pomocniczych należy zmienić funkcję **ge** na następujący schemat:

```

int ge(double *A, int SIZE)
{
    int k;
    int ii, jj;

    double packed_pivot[NC];
    double packed_mult[MC];

    for (k = 0; k < SIZE; k++) {
        double pivot = A[IDX(k, k, SIZE)];

        for (jj = k + 1; jj < SIZE; jj += NC) {
            int jb = MIN(NC, SIZE - jj);

            PackPivotRow(jb, &A[IDX(k, jj, SIZE)], packed_pivot);

            for (ii = k + 1; ii < SIZE; ii += MC) {
                int ib = MIN(MC, SIZE - ii);

                PackMultipliers(ib, A, SIZE, ii, k, pivot, packed_mult);

                UpdateKernel(ib, jb, &A[IDX(ii, jj, SIZE)],
                            SIZE, packed_pivot, packed_mult);
            }
        }
    }

    return 0;
}

```

Proszę skompilować procedurę z optymalizacją: Jaki jest czas działania?

Jakie GFLOP/s uzyskuje procesor?

2.7

Proszę wykonać kopię procedury

```
maciekw@Banach:~ lab3> cp ge6.c ge7.c
```

Proszę wprowadzić operacje wektorowe SSE3 (patrz laboratorium 2).

```
#include <x86intrin.h>

...
for (j = 0; j + NR <= n; j += NR) {
    register __m128d p01 = _mm_loadu_pd(pivot_row + j);
    register __m128d p23 = _mm_loadu_pd(pivot_row + j + 2);
    register __m128d p45 = _mm_loadu_pd(pivot_row + j + 4);
    register __m128d p67 = _mm_loadu_pd(pivot_row + j + 6);

    for (i = 0; i < m; i++) {
        double *c = C + i * ldc + j;

        register __m128d mmult = _mm_set1_pd(mult[i]);

        register __m128d c01 = _mm_loadu_pd(c);
        register __m128d c23 = _mm_loadu_pd(c + 2);
        register __m128d c45 = _mm_loadu_pd(c + 4);
        register __m128d c67 = _mm_loadu_pd(c + 6);

        c01 = _mm_sub_pd(c01, _mm_mul_pd(p01, mmult));
        c23 = _mm_sub_pd(c23, _mm_mul_pd(p23, mmult));
        c45 = _mm_sub_pd(c45, _mm_mul_pd(p45, mmult));
        c67 = _mm_sub_pd(c67, _mm_mul_pd(p67, mmult));

        _mm_storeu_pd(c, c01);
        _mm_storeu_pd(c + 2, c23);
        _mm_storeu_pd(c + 4, c45);
        _mm_storeu_pd(c + 6, c67);
    }
}
```

Proszę skompilować procedurę z optymalizacją.

Jaki jest czas działania?

Jakie GFLOP/s uzyskuje procesor?

2.8

Proszę wykonać kopię procedury

```
maciekw@Banach:~ lab3> cp ge7.c ge8.c
```

Proszę wprowadzić 256-bitowe operacje wektorowe AVX (patrz laboratorium 2).

```
#include <immintrin.h>
register __m256d ...
tmp0 = _mm256_loadu_pd (...);
...
```

Proszę skompilować procedurę z optymalizacją.

```
maciekw@Banach:~ lab3> gcc -mavx ge8.c
```

Jaki jest czas działania?

Jakie GFLOP/s uzyskuje procesor?

2.9

Proszę wszystkie uzyskane na Laboratoriach kody skompilować z opcją optymalizacji **-O2** oraz przetestować opcje **-march=native** oraz **-mfma**.

```
maciekw@Banach:~/studenci/OORA/skrypty/lab3> gcc -mavx -O2 ge7.c
```

```
maciekw@Banach:~/studenci/OORA/skrypty/lab3> ./a.out 1500
```

```
call GETime: 3.592460e-01
```

```
FLOP: 3.371626e+09
```

```
GFLOP/s: 9.385284e+00
```

```
Check: -4.605781e+16
```

```
maciekw@Banach:~ lab3> gcc -mavx -O2 -march=native ge8.c
```

```
maciekw@Banach:~ lab3> ./a.out 1500
```

```
call GETime: 4.534000e-01
```

```
FLOP: 3.371626e+09
```

```
GFLOP/s: 7.436316e+00
```

```
Check: -4.605781e+16
```

```
maciekw@Banach:~ lab3> gcc -mavx -O2 -mfma ge8.c
```

```
maciekw@Banach:~ lab3> ./a.out 1500
```

```
call GETime: 2.618760e-01
```

```
FLOP: 3.371626e+09
```

```
GFLOP/s: 1.287489e+01
```

```
Check: -4.605781e+16
```

```
maciekw@Banach:~ lab3> gcc -mavx -O2 -march=native -mfma ge8.c
```

```
maciekw@Banach:~ lab3> ./a.out 1500
```

```
call GETime: 2.518740e-01
```

```
FLOP: 3.371626e+09
```

```
GFLOP/s: 1.338616e+01
```

```
Check: -4.605781e+16
```

```
maciekw@Banach:~ lab3>
```

Jaki jest ich czas działania?

3 Podsumowanie

Które metody optymalizacji dały najlepsze rezultaty?
Dlaczego? Jakie mechanizmy za nimi stały?